

ENHANCEMENT-v2

Documentation Technique — JO2024 Ticketing Platform

Projet : Plateforme de billetterie Paris 2024

Stack API : Java 21 · Spring Boot 4.0 · PostgreSQL · JWT · Liquibase

Stack UI : Next.js 14 · TypeScript · Zustand · Tailwind CSS

Date : Avril 2026

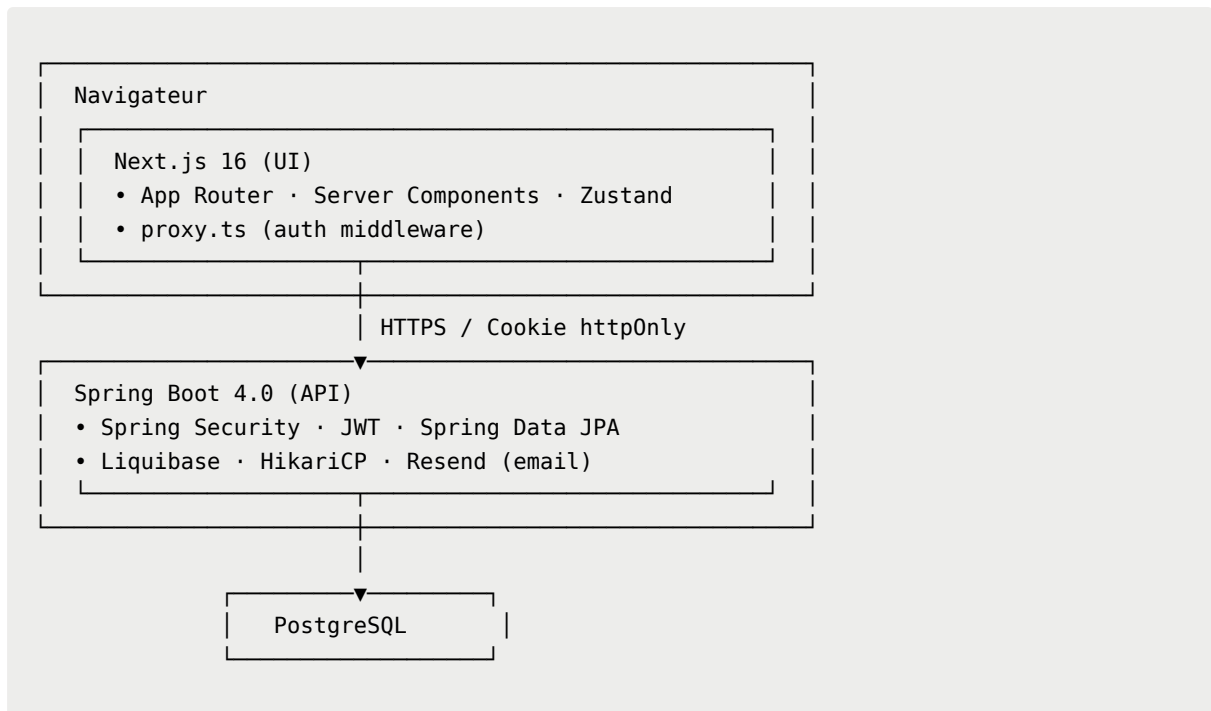
Auteur : Jocelyn Sainson

Table des matières

1. Vue d'ensemble de l'architecture
 2. Sécurité
 - 2.1 Vulnérabilités critiques
 - 2.2 Hardening de l'authentification
 - 2.3 Protection des données
 - 2.4 Sécurité réseau et transport
 - 2.5 Sécurité côté client (UI)
 3. Évolutions fonctionnelles
 - 3.1 Fiabilité transactionnelle
 - 3.2 Fonctionnalités utilisateur
 - 3.3 Administration et opérations
 4. Évolutions techniques
 - 4.1 Performance
 - 4.2 Observabilité
 - 4.3 Qualité du code et tests
 5. Roadmap priorisée
-

1. Vue d'ensemble de l'architecture

L'application est composée de deux couches indépendantes qui communiquent via une API REST :



Les sections suivantes documentent les problèmes de sécurité identifiés et les évolutions prévues, organisés par thème et par priorité d'implémentation.

2. Sécurité

2.1 Vulnérabilités critiques

Ces failles doivent être corrigées avant toute mise en production.

[SEC-CRIT-2] Absence de révocation des tokens JWT **API**

Couche : API · **Sévérité** : ● Critique

Localisation : `JwtAuthenticationFilter`, `AuthService`

Description

Le logout supprime le cookie côté client, mais le token d'accès (TTL 15 min) reste cryptographiquement valide jusqu'à son expiration naturelle. Un token intercepté ou volé ne peut pas être invalidé côté serveur.

Impact : Vol de session possible pendant 15 minutes après logout ou compromission d'un token.

Correctif

Maintenir une denylist Redis indexée par le claim `jti` (JWT ID) du token :

```
// À vérifier dans JwtAuthenticationFilter
if (tokenDenylistService.isRevoked(jwtId)) {
    throw new InvalidTokenException("Token has been revoke
d");
}
```

Lors du logout, ajouter le `jti` à la denylist avec un TTL égal à la durée de vie résiduelle du token.

[SEC-CRIT-3] Données d'authentification stockées dans localStorage **UI**

Couche : UI · **Sévérité** :  Critique

Localisation : `lib/stores/auth.store.ts`

Description

Le store Zustand persiste les données utilisateur (email, rôle, tokens) dans `localStorage` via `createJSONStorage(() => localStorage)`. Toute injection XSS permet d'extraire ces données et d'usurper l'identité de l'utilisateur.

Impact : Compromission complète de la session utilisateur par XSS.

Correctif

Déléguer entièrement la gestion de session aux cookies `httpOnly` produits par l'API. Côté client, ne conserver que les données d'affichage non sensibles (prénom, langue) sans persister le rôle.

[SEC-CRIT-4] Absence de rate limiting sur les endpoints sensibles **API**

Couche : API · **Sévérité** :  Critique

Localisation : `POST /api/auth/login`, `POST /api/2fa/send`, `POST /api/2fa/verify`, `POST /api/users/forget-password`

Description

Ces endpoints ne sont soumis à aucune limitation de débit. Un attaquant peut brute-forcer un mot de passe ou énumérer des codes OTP à la vitesse maximale de la machine, sans aucun délai ni blocage.

Impact : Compromission de comptes par force brute, contournement du 2FA.

Correctif

Intégrer Bucket4j comme filtre Spring Security :

Endpoint	Limite recommandée
POST /api/auth/login	5 tentatives / 60 s par IP
POST /api/2fa/send	3 envois / 5 min par utilisateur
POST /api/2fa/verify	5 tentatives / 10 min par utilisateur
POST /api/users/forget-password	3 requêtes / 15 min par IP

2.2 Hardening de l'authentification

[SEC-AUTH-1] Absence de verrouillage de compte sur les échecs de connexion API

Couche : API · **Sévérité :** ● Haute

Description

Le 2FA bloque correctement après 5 échecs, mais le formulaire de connexion par mot de passe ne trace aucune tentative échouée. Un attaquant peut tenter un nombre illimité de mots de passe sur un compte donné.

Correctif

Ajouter deux champs à l'entité `User` :

```
private int failedLoginAttempts;  
private LocalDateTime lockedUntil;
```

Appliquer un backoff exponentiel : 5 échecs → verrou 15 min, 10 échecs → verrou 1 h, 15 échecs → verrou permanent (déverrouillage manuel par admin).

[SEC-AUTH-2] Absence de protection CSRF sur les mutations UI

Couche : UI · **Sévérité :** ● Haute

Localisation : `lib/utils/apiClient.ts`

Description

Les requêtes mutantes (ajout au panier, checkout, modification de profil) s'appuient uniquement sur l'authentification par cookie. Sans token CSRF, un site tiers peut déclencher ces actions à l'insu de l'utilisateur connecté.

Correctif

Ajouter un endpoint `GET /api/csrf-token` côté API. Inclure ce token dans le header `X-CSRF-Token` de chaque requête mutante côté UI :

```
// lib/utils/apiClient.ts
headers: {
  "X-CSRF-Token": await getCsrfToken(),
}
```

[SEC-AUTH-3] Guards d'autorisation incomplets sur les routes admin **UI**

Couche : UI · **Sévérité :**  Moyenne

Localisation : `app/admin/`, `app/staff/`

Description

Le middleware `proxy.ts` protège les routes au niveau de la navigation, mais aucune vérification de rôle n'est effectuée au niveau des composants après un refresh de token. Un changement de rôle côté serveur n'est pas propagé immédiatement au client.

Correctif

Créer un hook `useRequiredRole(role: Role)` qui lit le rôle depuis le contexte d'authentification et redirige ou affiche une erreur si le rôle est insuffisant. Le coupler à une error boundary par segment de route.

[SEC-AUTH-4] Décodage JWT sans vérification de signature **UI**

Couche : UI · **Sévérité :**  Moyenne

Localisation : `proxy.ts`

Description

La fonction `decodeJwt()` de la librairie `jose` est utilisée pour lire les claims du token sans vérifier sa signature. Si la clé publique de l'API est accessible côté serveur Next.js, la signature devrait être vérifiée pour éviter d'accepter des tokens forgés.

Correctif

Ajouter la vérification de signature dans `proxy.ts` en utilisant `jwtVerify()` avec la clé publique RS256 de l'API. À défaut, documenter explicitement dans le code que la délégation à l'API est intentionnelle.

2.3 Protection des données

[SEC-DATA-1] Données personnelles (PII) stockées en clair

API

Couche : API · Sévérité : ● Haute

Description

L'email, le prénom et le nom des utilisateurs sont stockés en clair dans la base de données. Un dump de la base expose immédiatement toutes les données personnelles des clients, sans aucune barrière supplémentaire.

Correctif

Utiliser des JPA Attribute Converters avec chiffrement AES-256-GCM (via Spring Security Crypto) pour chiffrer ces colonnes de manière transparente :

```
@Convert(converter = EncryptedStringConverter.class)
private String email;
```

Pour préserver les lookups par email (authentification), indexer sur un hash déterministe HMAC-SHA256 de l'email dans une colonne séparée.

[SEC-DATA-2] Suppression physique des utilisateurs

API

Couche : API · Sévérité : ● Moyenne

Description

`userRepository.deleteById()` effectue un hard delete, orphelinant l'historique des transactions et des billets associés. Cela casse les pistes d'audit et peut créer des incohérences dans les données de billetterie.

Correctif

Implémenter un soft delete : ajouter `deleted_at TIMESTAMP` à la table `users`. Filtrer toutes les requêtes avec `WHERE deleted_at IS NULL` via un `@Where` Hibernate ou un repository custom. Seul un super-admin peut effectuer un hard delete après un délai légal.

[SEC-DATA-3] URL de réinitialisation de mot de passe hardcodée **API**

Couche : API · **Sévérité** : 🟡 Haute

Localisation : `UserService.resetPassword()`

Description

Le lien de réinitialisation intégré dans les emails est construit avec l'URL `http://localhost:3000/reset-password?token=...`. En production, les emails envoyés aux utilisateurs pointent vers localhost et le lien est donc non fonctionnel.

Correctif

Externaliser la base URL dans `application.yml` :

```
app:
  frontend-url: ${APP_FRONTEND_URL:http://localhost:3000}
```

```
@Value("${app.frontend-url}")
private String frontendUrl;
```

[SEC-DATA-4] Lacunes de conformité RGPD **API**

Couche : API · **Sévérité** : 🟡 Moyenne

Description

Aucun mécanisme ne permet aux utilisateurs d'exercer leurs droits RGPD : droit d'accès (Art. 15), droit à la portabilité (Art. 20) et droit à l'effacement (Art. 17).

Correctif

Implémenter deux endpoints dédiés :

- `GET /api/users/me/data-export` — génère un archive ZIP contenant toutes les PII, transactions et billets de l'utilisateur au format JSON/CSV.

- `DELETE /api/users/me` — anonymise les champs PII (remplacement par un hash), conserve les enregistrements de transaction pour les obligations comptables, et consigne l'action dans la piste d'audit.

2.4 Sécurité réseau et transport

[SEC-NET-1] Absence d'application HTTPS et d'en-têtes HSTS

API

Couche : API · Sévérité : ● Haute

Description

La configuration Spring Security ne redirige pas HTTP vers HTTPS et n'envoie pas d'en-tête `Strict-Transport-Security`. Le flag `Secure` sur les cookies seul est insuffisant si HTTP reste accessible.

Correctif

```
http.requiresChannel(channel ->
    channel.anyRequest().requiresSecure()
).headers(headers ->
    headers.httpStrictTransportSecurity(hsts ->
        hsts.maxAgeInSeconds(31536000).includeSubDomains(true)
    )
);
```

[SEC-NET-2] Absence d'en-têtes de sécurité HTTP

API

UI

Couche : API + UI · Sévérité : ● Moyenne

Description

Ni l'API ni l'UI n'émettent les en-têtes de sécurité HTTP recommandés (`Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`).

Correctif API — via le DSL `headers()` de Spring Security :

```
http.headers(headers -> headers
    .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'"))
```

```

    .frameOptions(frame -> frame.deny())
    .contentTypeOptions(Customizer.withDefaults())
    .referrerPolicy(referrer ->
        referrer.policy(ReferrerPolicyHeaderWriter.Referrer
            Policy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN)
        )
    );

```

Correctif UI — dans `next.config.ts` :

```

async headers() {
  return [{
    source: "/*",
    headers: [
      { key: "X-Content-Type-Options", value: "nosniff" },
      { key: "X-Frame-Options", value: "DENY" },
      { key: "Referrer-Policy", value: "strict-origin-when-
cross-origin" },
      { key: "Content-Security-Policy", value: "default-src
'self'; img-src 'self' data:; script-src 'self'" },
    ],
  }]
}

```

[SEC-NET-3] Version du SDK Resend non épinglée **API**

Couche : API · **Sévérité** :  Haute

Localisation : `pom.xml`

Description

`<version>LATEST</version>` rend les builds non reproductibles. Une release cassante de la dépendance peut casser silencieusement la CI ou introduire des régressions de sécurité sans alerte.

Correctif

Épingler à une version fixe et suivre les mises à jour de façon contrôlée :

```
<dependency>
  <groupId>com.resend</groupId>
  <artifactId>resend-java</artifactId>
  <version>3.1.0</version> <!-- version fixe -->
</dependency>
```

Activer Dependabot ou Renovate pour les alertes de mise à jour.

2.5 Validation des entrées

[SEC-INPUT-1] Validation manquante sur plusieurs DTOs API

Couche : API · **Sévérité** : ● Haute

Description

Plusieurs DTOs acceptent des valeurs invalides ou dangereuses faute de contraintes de validation Bean Validation :

DTO	Champ	Contrainte manquante
CheckoutRequestDTO	cvv	@Pattern(regexp="^[0-9]{3,4}\$")
CheckoutRequestDTO	cardNumber	Valdateur Luhn custom
CheckoutRequestDTO	expiryMonth/Year	Date future uniquement
ChangePasswordRequestDTO	newPassword	@Size(min=8, max=50) + @NotBlank
OfferRequestDTO	price	@DecimalMin("0.01")
EventRequestDTO	capacity	@Positive
EventRequestDTO	availableSlots	≤ capacity (@AssertTrue au niveau classe)

Correctif

Ajouter les annotations Bean Validation sur chaque champ et s'assurer que @Valid est présent sur les paramètres de contrôleur correspondants.

[SEC-INPUT-2] Numéros de carte de test exposés dans l'UI UI

Couche : UI · **Sévérité** : ● Moyenne

Localisation : app/checkout/page.tsx

Description

Des numéros de carte de test sont affichés et cliquables dans l'interface de paiement. En production, cette information peut être exploitée à des fins de test frauduleux ou nuire à la confiance des utilisateurs.

Correctif

```
{process.env.NODE_ENV !== 'production' && (  
  <TestCardsHelper />  
)}
```

Ou supprimer l'affichage et les déplacer dans la documentation interne de développement.

3. Évolutions fonctionnelles

3.1 Fiabilité transactionnelle

[FEAT-TRANS-1] Idempotence du checkout **API**

Couche : API · **Priorité** : Haute

Contexte

Une réessai réseau ou un double-clic sur `POST /api/checkout` crée une transaction dupliquée et débite la carte deux fois. Ce cas d'usage est courant en conditions réelles (réseau mobile instable, timeout client).

Solution proposée

Accepter un header `Idempotency-Key` fourni par le client (UUID v4) et mettre en cache la réponse dans Redis pendant 24 h :

```
POST /api/checkout  
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

Un second appel avec la même clé retourne la réponse en cache sans retraiter le paiement.

[FEAT-TRANS-2] Découplage de la livraison des emails **API**

Couche : API · **Priorité** : Haute

Contexte

`EmailService` est appelé directement dans des méthodes `@Transactional` de `TransactionService`. Si le service Resend est lent ou indisponible, la réponse HTTP de checkout se bloque, voire échoue — après que le paiement a déjà été prélevé.

Solution proposée

Découpler l'envoi d'emails par événements Spring :

```
// Dans TransactionService
eventPublisher.publishEvent(new OrderConfirmedEvent(transaction));

// Dans EmailEventListener (composant séparé)
@EventListener
@Async
public void onOrderConfirmed(OrderConfirmedEvent event) {
    emailService.sendConfirmation(event.getTransaction());
}
```

Pour un volume élevé, migrer vers une file RabbitMQ ou Kafka avec retry automatique.

[FEAT-TRANS-3] Flux d'annulation et de remboursement API

Couche : API · **Priorité** : Haute

Contexte

`TransactionStatus` ne contient pas de valeur `CANCELLED` ou `REFUNDED`. Il est impossible d'annuler une commande ou d'invalider des billets après achat.

Solution proposée

Étendre `TransactionStatus` : `PENDING`, `COMPLETED`, `CANCELLED`, `REFUNDED`.

Ajouter l'endpoint :

```
POST /api/transactions/{id}/cancel
```

Accessible à l'admin et au propriétaire de la transaction. L'action :

1. Marque tous les billets liés `isValid = false`
2. Met à jour `TransactionStatus` → `CANCELLED`

3. Libère les `availableSlots` sur les événements concernés
 4. Déclenche le remboursement via `PaymentGateway`
 5. Envoie une confirmation par email
-

[FEAT-TRANS-4] Verrouillage optimiste sur la réservation de places **API**

Couche : API · **Priorité** : Haute

Contexte

Deux requêtes simultanées `POST /api/cart/add` pour le même événement peuvent toutes deux lire `availableSlots > 0` et procéder, créant un état de surréservation non détectable.

Solution proposée

Ajouter le verrouillage optimiste JPA :

```
@Version
private Long version;
```

Sur les entités `Event` et `Cart`. Capturer `OptimisticLockException` dans les services concernés et retourner HTTP 409 avec un message explicite invitant à réessayer.

3.2 Fonctionnalités utilisateur

[FEAT-USER-1] Indicateurs de stock et gestion sold out **UI**

Couche : UI · **Priorité** : Moyenne

Contexte

L'interface n'indique pas le nombre de places disponibles, et les boutons d'ajout au panier restent actifs même lorsqu'une offre est épuisée.

Solution proposée

- Afficher le stock restant depuis les données d'offre (`availableSlots`)
 - Afficher un bandeau "Plus que X places disponibles" sous un seuil configurable (ex. < 10)
 - Remplacer le bouton "Ajouter" par "Complet" désactivé si `availableSlots === 0`
-

[FEAT-USER-2] Pagination et filtres avancés dans le catalogue

API **UI**

Couche : API + UI · **Priorité** : Moyenne

Contexte

Les endpoints `GET /api/events`, `GET /api/offers`, `GET /api/sports` retournent tous les enregistrements sans pagination. Côté UI, le filtrage est limité aux catégories de sport.

Solution proposée (API)

Accepter les paramètres `Pageable` de Spring Data :

```
GET /api/events?page=0&size=20&sort=startDate,asc&sportId=3&available=true
```

Solution proposée (UI)

Ajouter des filtres multi-critères (date, sport, lieu, fourchette de prix) en URL search params pour permettre le partage des résultats filtrés.

[FEAT-USER-3] Système de codes promotionnels **UI** **API**

Couche : UI + API · **Priorité** : Moyenne

Contexte

Aucun mécanisme de réduction n'existe. Il n'est pas possible d'offrir des promotions lors d'événements commerciaux ou pour des partenaires.

Solution proposée

Ajouter un champ code promo dans l'interface de checkout, validé par un endpoint dédié :

```
POST /api/promo/validate
{ "code": "PARIS2024", "cartId": "..." }
```

Le backend calcule et applique la remise, retournée dans le récapitulatif du panier.

[FEAT-USER-4] Relance sur abandon de panier **API**

Couche : API · **Priorité** : Basse

Contexte

Les paniers expirent silencieusement. Les utilisateurs ayant manifesté un intérêt sans finaliser l'achat ne reçoivent aucune relance.

Solution proposée

Ajouter un job planifié Spring Scheduler :

```
@Scheduled(fixedDelay = 900000) // toutes les 15 min
public void notifyAbandonedCarts() {
    // Carts expirant dans < 1h et dont l'utilisateur n'a p
    as reçu de relance
}
```

3.3 Administration et opérations

[FEAT-ADMIN-1] Piste d'audit sur les opérations sensibles **API**

Couche : API · **Priorité** : Haute

Contexte

Aucun enregistrement ne trace qui a modifié un rôle utilisateur, scanné un billet, déclenché une réinitialisation de mot de passe ou annulé une transaction. Cette traçabilité est une exigence de conformité pour les événements régulés.

Solution proposée

Activer Hibernate Envers sur les entités critiques :

```
@Entity
@Audited
public class User { ... }
```

Compléter avec Spring Data Auditing (`@CreatedBy` , `@LastModifiedBy`) alimenté par le principal JWT courant via `AuditorAware<String>` .

[FEAT-ADMIN-2] Intégration d'une passerelle de paiement réelle **API**

Couche : API · **Priorité** : Moyenne

Contexte

`MockPaymentGateway` approuve les cartes par des patterns hardcodés. L'interface `PaymentGateway` est correctement abstraite et prête pour une implémentation réelle.

Solution proposée

Implémenter `StripePaymentGateway` activé par propriété :

```
@ConditionalOnProperty(name = "payment.provider", havingValue = "stripe")
public class StripePaymentGateway implements PaymentGateway
{ ... }
```

Utiliser l'API Payment Intents de Stripe. Ne jamais transiter les numéros de carte bruts — utiliser la tokenisation côté client Stripe.js + `paymentMethodId`.

4. Évolutions techniques

4.1 Performance

[TECH-PERF-1] Rendu statique et ISR sur les routes publiques UI

Couche : UI · **Priorité** : Haute

Contexte

Toutes les routes utilisent le rendu dynamique par défaut, alors que le catalogue d'événements, les sports et les offres sont quasi-statiques.

Solution proposée

```
// app/events/page.tsx
export const revalidate = 300; // ISR toutes les 5 min

// app/events/[id]/page.tsx
export async function generateStaticParams() {
  const events = await getEvents();
  return events.map(e => ({ id: e.id }));
}
export const revalidate = 600;
```

[TECH-PERF-2] Mise en cache des données de référence API

Couche : API · **Priorité** : Moyenne

Contexte

Événements, sports et offres sont interrogés à chaque requête sans cache, générant des requêtes redondantes sur des données rarement modifiées.

Solution proposée

Activer Spring Cache avec Caffeine (local) :

```
@Cacheable("events")
public List<EventResponseDTO> getAllEvents() { ... }

@CacheEvict(value = "events", allEntries = true)
public EventResponseDTO createEvent(EventRequestDTO dto) {
    ...
}
```

[TECH-PERF-3] Configuration du pool de connexions HikariCP API

Couche : API · **Priorité** : Moyenne

Contexte

Spring Boot utilise les valeurs HikariCP par défaut (10 connexions max). Sous charge, le pool s'épuise avant que la base de données ne soit le facteur limitant.

Solution proposée

```
spring:
  datasource:
    hikari:
      maximum-pool-size:20
      minimum-idle:5
      idle-timeout:30000
      max-lifetime:1800000
      connection-timeout:20000
```

[TECH-PERF-4] Index de base de données manquants API

Couche : API · **Priorité** : Moyenne

Contexte

Les colonnes suivantes sont interrogées sur chaque chemin critique sans index explicite dans les changesets Liquibase :

Table	Colonne(s)	Chemin critique
<code>tickets</code>	<code>combined_key</code>	Scan de billet
<code>tickets</code>	<code>barcode</code>	Scan de billet
<code>users</code>	<code>email</code>	Authentification
<code>two_factor_codes</code>	<code>user_id, expires_at</code>	Vérification OTP
<code>carts</code>	<code>user_id, status</code>	Lookup panier actif
<code>transactions</code>	<code>payment_reference</code>	Résolution de litiges

Correctif

Ajouter des changesets `<createIndex>` dans Liquibase pour chacune de ces colonnes.

[TECH-PERF-5] Déduplication des requêtes panier et états de chargement **UI**

Couche : UI · **Priorité** : Moyenne

Contexte

`CartInitializer.tsx` recharge le panier à chaque montage sans déduplication. En parallèle, le checkout affiche `null` pendant le chargement sans indicateur visuel.

Solution proposée

- Déplacer l'initialisation du panier dans le root layout avec un flag d'initialisation
- Ajouter des `<Suspense>` boundaries et des skeleton loaders sur les routes critiques
- Utiliser `useTransition()` pour les soumissions de formulaires afin de maintenir l'UI réactive

4.2 Observabilité

[TECH-OBS-1] Couverture de logs insuffisante **API**

Couche : API · **Priorité** : Moyenne

Contexte

Seulement 8 classes sur 60+ ont `@Slf4j`. Des classes critiques comme `CartService`, `UserService`, `JwtService`, `TwoFactorService` ne produisent aucun log, rendant le debugging en production difficile.

Solution proposée

Ajouter des logs structurés à toutes les classes service et filter. Configurer Logback avec un encodeur JSON pour le profil de production. Logger systématiquement les événements de sécurité en `WARN` : échec d'authentification, échec OTP, tentative de changement de rôle.

[TECH-OBS-2] Indicateurs de santé custom **API**

Couche : API · **Priorité** : Basse

Contexte

`/actuator/health` vérifie uniquement la connectivité JVM et base de données. Il ne contrôle pas la disponibilité du service email (Resend) ni l'espace disque pour la génération de PDFs.

Solution proposée

```
@Component
public class ResendHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        // Ping léger vers l'API Resend
        return resendClient.isReachable() ? Health.up().build() : Health.down().build();
    }
}
```

4.3 Qualité du code et tests

[TECH-QA-1] Absence de tests côté UI **UI**

Couche : UI · **Priorité** : Haute

Contexte

Aucun framework de test n'est configuré côté UI. La CI se limite au lint, au build et à l'audit de dépendances.

Plan de mise en place

Étape 1 — Tests unitaires avec Vitest + React Testing Library

```
pnpm add -D vitest @testing-library/react @testing-library/user-event jsdom
```

Ordre de priorité des tests :

1. Schémas de validation Zod — `lib/schemas/`
2. Gestion des erreurs API — `lib/utils/apiClient.ts`
3. Mutations panier — `lib/stores/cart.store.ts`
4. Flow de paiement — `app/checkout/`

Étape 2 — Tests E2E avec Playwright

```
pnpm add -D @playwright/test
```

Parcours critiques à couvrir :

- Inscription → Connexion → 2FA → Catalogue → Panier → Checkout → Confirmation
- Scan QR code (profil staff)
- Gestion des événements (profil admin)

[TECH-QA-2] Coverage gates trop restrictifs côté API **API**

Couche : API · **Priorité** : Basse

Contexte

Le seuil JaCoCo de 80% ne couvre que `dev.jos.back.service`. Les contrôleurs, mappers et utilitaires échappent aux gates de couverture.

Solution proposée

Étendre les règles JaCoCo :

- `dev.jos.back.controller` : seuil $\geq 70\%$
- `dev.jos.back.mapper` : seuil $\geq 90\%$ (fonctions pures, faciles à tester)

[TECH-QA-3] Error boundaries absentes **UI**

Couche : UI · **Priorité** : Haute

Contexte

Aucun fichier `error.tsx` n'existe dans l'application. Une exception non gérée dans un composant fait crasher la page entière et expose potentiellement une stack trace à l'utilisateur.

Solution proposée

Créer des error boundaries à plusieurs niveaux :

- `app/error.tsx` — boundary racine
- `app/admin/error.tsx` — boundary segment admin
- `app/checkout/error.tsx` — boundary segment paiement

Chaque boundary affiche une UI d'erreur conviviale avec un bouton "Réessayer" et journalise l'erreur vers un service de monitoring.

[TECH-QA-4] Validation des variables d'environnement **UI**

Couche : UI · **Priorité** : Basse

Contexte

Aucune vérification que les variables d'environnement requises sont définies au démarrage. Une variable manquante provoque des erreurs silencieuses à l'exécution plutôt qu'un échec clair au démarrage.

Solution proposée

```
// env.ts
import { z } from "zod";

const envSchema = z.object({
  NEXT_PUBLIC_API_URL: z.string().url(),
  NEXTAUTH_SECRET: z.string().min(32),
});

export const env = envSchema.parse(process.env);
```

5. Roadmap priorisée

Phase 1 — Corrections bloquantes (avant mise en production)

ID	Couche	Titre	Sévérité
SEC-CRIT-2	API	Révocation JWT impossible	 Critique
SEC-CRIT-3	UI	Auth data dans localStorage	 Critique
SEC-CRIT-4	API	Absence de rate limiting	 Critique
FEAT-TRANS-4	API	Race condition sur réservation de places	 Haute
SEC-INPUT-1	API	Validation DTO manquante	 Haute

Phase 2 — Hardening sécurité (sprint 1-2)

ID	Couche	Titre	Sévérité
SEC-AUTH-1	API	Verrouillage de compte	 Haute
SEC-AUTH-2	UI	Protection CSRF	 Haute
SEC-DATA-1	API	Chiffrement PII	 Haute
SEC-NET-1	API	HTTPS / HSTS	 Haute
SEC-NET-3	API	Version SDK Resend	 Haute
SEC-DATA-3	API	URL reset hardcodée	 Haute
FEAT-TRANS-1	API	Idempotence checkout	 Haute
FEAT-TRANS-2	API	Emails asynchrones	 Haute
FEAT-TRANS-3	API	Flux annulation/remboursement	 Haute
FEAT-ADMIN-1	API	Piste d'audit	 Haute
TECH-QA-1	UI	Tests Vitest + Playwright	 Haute
TECH-QA-3	UI	Error boundaries	 Haute
TECH-PERF-1	UI	ISR routes publiques	 Haute
FEAT-USER-1	UI	Mes billets	 Haute

Phase 3 — Évolutions fonctionnelles et performance (sprint 3-5)

ID	Couche	Titre	Sévérité
SEC-NET-2	API+UI	En-têtes de sécurité HTTP	 Moyenne
SEC-DATA-2	API	Soft delete utilisateurs	 Moyenne
SEC-DATA-4	API	Conformité RGPD	 Moyenne
SEC-AUTH-3	UI	Guards admin complets	 Moyenne

ID	Couche	Titre	Sévérité
SEC-AUTH-4	UI	Vérification signature JWT	● Moyenne
SEC-INPUT-2	UI	Cartes de test en production	● Moyenne
FEAT-USER-2	UI	Indicateurs de stock	● Moyenne
FEAT-USER-3	API+UI	Pagination et filtres avancés	● Moyenne
FEAT-USER-4	API+UI	Codes promotionnels	● Moyenne
FEAT-ADMIN-2	API	Passerelle de paiement réelle	● Moyenne
TECH-PERF-2	API	Cache Spring (Caffeine)	● Moyenne
TECH-PERF-3	API	HikariCP configuré	● Moyenne
TECH-PERF-4	API	Index base de données	● Moyenne
TECH-PERF-5	UI	Suspense / skeletons	● Moyenne
TECH-OBS-1	API	Logs structurés complets	● Moyenne
TECH-QA-2	API	Coverage gates élargis	● Moyenne

Phase 4 — Améliorations continues (backlog)

ID	Couche	Titre	Sévérité
FEAT-USER-5	API	Relance abandon de panier	● Basse
TECH-OBS-2	API	Indicateurs de santé custom	● Basse
TECH-QA-4	UI	Validation variables d'env	● Basse